WPYEG · Edmonton WordPress Meetup

Better by Default

Secure defaults for every WordPress site.

a hands-on workshop · build the “sane-defaults” plugin

WordPress is open by default; hosts vary in what they close.

None of these are bugs — or quite the security risks popular opinion alleges. They are defaults chosen for maximum compatibility on a 20+ year-old web application.

Usernames leak

REST and author archives expose public author slugs that often resemble login names.

XML-RPC exposed

A venerable interop protocol Jetpack still uses. If you don't need it, it just adds attack surface.

Dead weight loads

Emoji scripts, version tags, and RSD links on every page. Do you need them?

Spam surface invites

Comments, pingbacks, and trackbacks are open by default. Legacy cruft, or the heart of the open web?

The one idea to take home

A “default” is just an opinionated filter behind a toggle.

```
if ( wpyeg_defaults_enabled( 'restrict_rest_user_discovery' ) ) {
    add_filter( 'rest_endpoints', $hide_users_endpoint );
}   // that's the whole pattern, repeated across the plugin
```

Hooks

WordPress is built to be interrupted at labelled moments (hooks) so you never edit core code.

A

Actions

“When you reach this moment, also DO this.”

```
add_action( 'init', 'my_callback' );
```

F

Filters

“Before you use this value, let me CHANGE it first.”

```
add_filter( 'xmlrpc_enabled',
'__return_false' );
```

↓ What wins when settings overlap?

1	**wp-config.php constants**	Load first. When core treats one as authoritative, plugin settings cannot override it.
2	**Must-use plugins**	Load before normal plugins, so their callbacks register first.
3	**Normal plugins**	Load in active_plugins order — AX before BBD on this demo site.
4	**Hook priority**	Lower runs earlier; higher runs later. Ties keep registration order.

```
effective behaviour = hard constants + every callback, in execution order
```

Eight categories of defaults

1

Security

Make the attack surface smaller

2

Updates

A deliberate core & translation policy

3

Content

Close spam channels & info leaks

4

Admin UX

A calmer, faster, prettier dashboard

5

Login

Sessions & credentials

6

Branding

Own your login screen

7

Performance

Trim the fat

8

Email

Say so when the site cannot send mail

1

SECTION ONE

Security & Attack Surface

Every item here removes something an attacker can poke — usually in one line.

Restrict REST API user discovery

The /wp/v2/users endpoint hands out every author's login name to anyone — half of a brute-force guess, for free. Closing it for logged-out requests only keeps the editor and legit integrations working.

```
KEY       restrict_rest_user_discovery
DEFAULT
yes
```

```
add_filter( 'rest_endpoints', function ( $ep ) {
    if ( ! is_user_logged_in() ) {
        unset( $ep['/wp/v2/users'] );
        unset(
          $ep['/wp/v2/users/(?P<id>[\d]+)']
        );
    }
    return $ep;
} );
```

Lock REST to logged-in users (opt-in)

The sledgehammer version of the slide before: requiring auth for ALL REST calls stops anonymous scraping cold. It breaks anonymous REST — front-end blocks, embeds, search, outside integrations — so it ships off.

```
KEY      disable_rest
DEFAULT
no
```

```
add_filter( 'rest_authentication_errors',
  function ( $result ) {
    // Only an ERROR short-circuits. A cookie with no
    // nonce resolves to user 0 and returns true.
    if ( is_wp_error( $result ) ) return $result;
    // Leave oEmbed open, or every site embedding
    // yours silently degrades to a bare link.
    if ( route_is_public() ) return $result;
    if ( ! is_user_logged_in() ) {
      return new WP_Error(
        'rest_not_logged_in', 'Auth required.',
        array( 'status' => 401 ) );
    }
    return $result;
} );
```

Lock XML-RPC down by category

XML-RPC is legitimate but aging — an extra surface, not a backdoor. We unplug unused method families while preserving the endpoint for integrations that need it.

```
// each category off → remove its methods
add_filter( 'xmlrpc_methods', function ( $m ) {
  if ( ! allow( 'pingbacks' ) )
    unset( $m['pingback.ping'],
           $m['pingback.extensions.getPingbacks'] );
  if ( ! allow( 'remote_publishing' ) )
    // drop wp.* metaWeblog.* mt.* blogger.*
  return $m;
} );

// multicall can't be filtered off (IXR re-adds it)
// → swap in a server that refuses it
add_filter( 'wp_xmlrpc_server_class', $refuse );
```

KEY	DEFAULT
xmlrpc allow keys + block_xmlrpc_endpoint	no (all four)

Keep Application Passwords available

This is an existing default we don't lock down. An Application Password is like a spare key cut for one app: hashed, per-application, revocable on its own — the safer REST credential, and the only one core accepts for REST Basic Auth.

```
KEY      disable_application_passwords
DEFAULT
no (available)
```

```
// available by default —
// prohibit only if opted in
if ( wpyeg_defaults_enabled(
        'disable_application_passwords' ) ) {
  add_filter(
    'wp_is_application_passwords_available',
    '__return_false'
  );
}
```

Screen breaches without sending the password

NIST SP 800-63B-4 § 3.1.1.2: use 15+ characters, a blocklist, and no composition rules. BBD hashes the candidate locally, sends HIBP only the first 5 SHA-1 characters, and matches the remaining 35 locally. The password and full hash never leave WordPress; unavailable or invalid HIBP data fails open.

```
KEY       require_strong_passwords
DEFAULT
yes
```

```
$hash   = strtoupper( sha1( $pw ) );
$prefix = substr( $hash, 0, 5 );
$suffix = substr( $hash, 5 );

// HIBP receives $prefix and returns
// matching suffixes plus breach counts.
// BBD compares $suffix locally.
if ( response_contains( $suffix ) ) {
    reject( 'Seen in a breach.' );
}
// invalid or 128 KiB => fail open
```

Remove fingerprints, add headers

Two headers with no real downside ship on; framing is its own setting, because it is the only one that can break a working site. Hiding the version is obscurity, not hardening, so it ships off.

```
KEY       security_headers / frame_options
DEFAULT
yes / SAMEORIGIN
```

```php
remove_action( 'wp_head', 'wp_generator' );

add_filter( 'wp_headers', function ( $h ) {
  // compare, do not yield - see the notes
  $key = find_key( $h, 'X-Frame-Options' );
  if ( stronger( $want, $h[ $key ] ) )
    $h[ $key ] = $want;

  // one effective value, so correct it
  $h[ ctk( $h ) ] = 'nosniff';
  return $h;
} );
```

The filter that calls itself

Editors hold unfiltered_html on a single-site install — enough to save a raw <script> into a post. Taking it back is easy. Doing it without blowing the stack is the lesson.

```
KEY       limit_unfiltered_html_to_admins
DEFAULT
yes
```

```
add_filter( 'user_has_cap', function (
    $allcaps, $caps, $args, $user ) {

  if ( empty( $allcaps['unfiltered_html'] ) ) {
    return $allcaps;
  }

  $roles = isset( $user->roles )
    ? (array) $user->roles : array();

  // Read what you were handed. Never ask.
  if ( in_array( 'administrator', $roles, true )
    || ! empty( $allcaps['manage_options'] ) ) {
    return $allcaps;
  }

  $allcaps['unfiltered_html'] = false;
  return $allcaps;
}, PHP_INT_MAX - 1, 4 );
```

Decide who patches WordPress

Core ships maintenance and security releases automatically already. The question this default answers is not whether to allow that — it is whether YOU are deciding, or whether your host quietly decided for you and never said.

```
KEY       core_update_policy
DEFAULT
minor
```

```php
// Two filters, because core asks two questions.
add_filter( 'allow_minor_auto_core_updates', $f );
add_filter( 'allow_major_auto_core_updates', $g );

// 'inherit' hands the value straight back — the
// setting that chooses NOT to have an opinion.
if ( 'inherit' === $policy ) { return $enabled; }

return in_array( $policy, [ 'minor', 'all' ], true );
```

2

SECTION TWO

Content & Public Surfaces

Close the spam funnels and the thin pages Google (and bots) love to crawl.

Disable comments, trackbacks & pingbacks

For many sites, comments are a spam magnet with little upside. Here we close them everywhere, hide existing threads, and drop the admin menu.

```
KEY      disable_comments / disable_pingbacks /
disable_self_pingbacks
DEFAULT
yes each
```

```
add_filter( 'comments_open', '__return_false', 20 );
add_filter( 'pings_open',    '__return_false', 20 );
add_filter( 'comments_array',
            '__return_empty_array', 20 );
add_filter( 'get_comments_number', '__return_zero', 20 );
add_filter( 'comments_pre_query',
            $empty_comment_queries, 10, 2 );
add_filter( 'render_block',
            $suppress_comment_blocks, 10, 2 );

// + remove_post_type_support() on init
// + remove_menu_page('edit-comments.php')
// + drop the admin-bar comments node
```

A clean 404 needs redirect_canonical gone

Dropping the feed link stops the feed being advertised, not served. Answering it takes a 404 — and the 404 takes one more removal that no filter-level test will ever catch.

```
KEY       disable_comments
DEFAULT
yes
```

```
add_action( 'template_redirect',
            $block_comment_feeds, 9 );

function block_comment_feeds() {
  if ( ! is_comment_feed() ) { return; }

  $wp_query->set_404();
  remove_action( 'template_redirect',
                 'redirect_canonical' );

  status_header( 404 );
  nocache_headers();
}
```

Redirect author & attachment pages

Author archives expose the authors' usernames in the URL, and attachment pages are near-empty media wrappers. Both dilute SEO and are targets for trouble. Same hook, two conditions.

```
add_action( 'template_redirect', function () {
  if ( is_author() ) {
    wp_safe_redirect( home_url('/'), 301 );
    exit;
  }
  if ( is_attachment() ) {
    // parent post, else the FILE - never home
  }
} );
```

KEY	DEFAULT
disable_author_archives / redirect_attachment_pages	yes / yes

Disable the emoji script

WordPress core injects an emoji-detection script and inline CSS on every page load, plus a DNS-prefetch hint. Modern browsers render emoji natively, so this is pure dead weight.

```
KEY       disable_emojis
DEFAULT
yes
```

```
add_action( 'init', function () {
  remove_action( 'wp_head',
    'print_emoji_detection_script', 7 );
  remove_action( 'wp_print_styles',
    'print_emoji_styles' );
  // ...admin + feed + mail variants too
  add_filter( 'emoji_svg_url',
              '__return_false' );
} );
```

3

SECTION THREE

Admin UX & Login Sessions

Small quality-of-life defaults: a calmer dashboard and sensible session policy.

ADMIN UX

Faster search, quieter admin bar

Search the admin post list on a big site and WordPress reads every word of every post — like finding a book by reading the whole library. Title-only search checks just the spines, and it's far faster.

```
KEY      title_only_admin_search /
frontend_admin_bar_behavior
DEFAULT
no / ''
```

```php
// title-only admin search — narrow the COLUMNS
add_filter( 'post_search_columns',
  function ( $cols, $s, $q ) {
    if ( is_admin() && $q->is_main_query() )
        return array( 'post_title' );
    return $cols;   // front-end untouched
  }, 10, 3 );
// hide bar for non-admins
add_filter( 'show_admin_bar', fn( $s ) =>
  current_user_can('manage_options') ? $s : false );
```

Right-size the login session

A normal login lasts 2 days; “Remember Me” extends it to 14. Both are in days, and the remembered one can never be shorter. Now look at the registration: this callback throws away the value it was handed, so two plugins setting it cannot both win — and at core's own 2 / 14 we have nothing to say, so we stay out of it.

```
function session_length( $exp, $uid, $remember ) {
  return $remember
    ? 14 * DAY_IN_SECONDS
    : 2 * DAY_IN_SECONDS;
}

// only when we differ from core's 2 / 14,
// and late enough to be the last word
if ( policy_is_custom() )
  add_filter( 'auth_cookie_expiration',
    'session_length', 50, 3 );
```

```
KEY       disable_remember_me /
session_regular_days / remember_me_days
DEFAULT
no / 2 / 14
```

4

SECTION FOUR

Branding, Performance & Email

Brand the login screen, throttle one polling API, and end on the only default that changes nothing at all.

BRANDING

Own the login screen

The default WordPress logo sends users to wordpress.org. Removing, unlinking, or replacing it keeps the login screen organizationally consistent and prevents an unexpected external destination. BBD leaves it unchanged unless you opt in.

```
KEY      login_logo_behavior
DEFAULT
keep_default (keep / remove / unlink / replace)
```

```php
// remove, unlink, or replace — a choice
add_action( 'login_head', $logo_css );

// any change points the link home
// (no separate toggle)
add_filter( 'login_headerurl', 'home_url' );
add_filter( 'login_headertext', fn() =>
            get_bloginfo( 'name' ) );
```

Throttle Heartbeat — and a default we deleted

Throttle Heartbeat to ease up on weak shared hosting. The more interesting half is the toggle that used to be here: WordPress 6.3 gave scripts a per-script loading strategy, so our blanket defer filter had to go.

```
KEY       throttle_heartbeat
DEFAULT
no (opt-in)
```

```php
add_filter( 'heartbeat_settings', function ( $s ) {
  $s['interval'] = 60;
  return $s;
} );

// Deferring is NOT a setting here. Since WP 6.3:
wp_enqueue_script( 'front', $src, array(), '1.0',
  array( 'strategy' => 'defer' ) );
```

EMAIL

Say so when the site cannot send mail

WordPress sends from wordpress@yourdomain unless something changes it. On a domain that cannot send, password resets fail silently — wp_mail() returns false and nothing surfaces it. This warns, and that is all it does.

```
// The From address the site will ACTUALLY use.
$from = apply_filters( 'wp_mail_from',
          'wordpress@' . $host );

// A shape check, not a delivery test.
$risky = ! is_email( $from )
  || in_array( $domain, $never_resolves )
  || preg_match( '/\.(local|test|invalid)$/', $d );

// Warn. Never block, never rewrite.
add_action( 'admin_notices', $render_notice );
```

```
KEY       mail_deliverability_notice
DEFAULT
yes
```

Three things a plugin can't toggle

Some defaults live in wp-config.php, above the plugin layer, because they must load before plugins do. Document them as manual steps in your onboarding checklist.

```
define( 'DISALLOW_FILE_EDIT', true );  // no in-dashboard code editor
define( 'AUTOSAVE_INTERVAL', 120 );    // gentler autosave (seconds)
define( 'WP_POST_REVISIONS', 10 );     // cap revision-table bloat
```

1 Kills the theme/plugin editor

A stolen admin login can't rewrite your PHP.

2 Writes to the DB less often

Fewer autosave revisions during long edits.

3 Keeps revisions in check

Ten per post instead of unbounded growth.

How the plugin is built

One array is the whole map. Read it and you understand the plugin.

1 `schema()`

One array: every setting, its default, type & group. The single source of truth.

2 `settings page`

Loops the schema to render toggles under Settings → Better by Default.

3 `bootstrap()`

For each ENABLED key, wires its add_filter / add_action to the right hook.

```
$stored = get_option('wpyeg_better_by_default');   // read once
foreach ( wpyeg_defaults_schema() as $key => $field ) { /* render + wire */ }
```

Hands-on: install & flip switches

1. **Upload the plugin** — Plugins → Add New → Upload Plugin → choose sane-defaults.zip → Activate.
2. **Open the settings** — Settings → Better by Default; every toggle grouped by category.
3. **Verify a default** — Visit /wp-json/wp/v2/users logged out → 401 or empty, not a list of usernames.
4. **Toggle & re-check** — Flip a switch off, reload, watch the behaviour change.

prefer the terminal? `wp plugin install ./sane-defaults.zip --activate`

Your turn

Add one new default to the plugin

Goal: disable the WordPress dashboard "Welcome" panel. Two small edits — no new settings-page code.

```
// 1) add a schema entry in wpyeg_defaults_schema()
'hide_welcome_panel' => array(
    'default' => 'yes', 'type' => 'toggle', 'group' => 'ux',
    'label' => 'Hide dashboard welcome panel',
),

// 2) wire it inside wpyeg_defaults_bootstrap()
if ( wpyeg_defaults_enabled( 'hide_welcome_panel' ) ) {
    remove_action( 'welcome_panel', 'wp_welcome_panel' );
}
```

Schema map — security surfaces and credentials

Exact keys and defaults from wpyeg_defaults_schema().

Setting key or owner	Default	Core hook / authority
restrict_rest_user_discovery	yes	rest_endpoints
disable_rest	no	rest_authentication_errors
xmlrpc_allow_pingbacks	no	xmlrpc_methods / headers
xmlrpc_allow_remote_publishing	no	xmlrpc_methods / discovery
xmlrpc_allow_multicall	no	wp_xmlrpc_server_class
block_xmlrpc_endpoint	no	template_redirect
disable_application_passwords	no	wp_is_application_passwords_available
require_strong_passwords	yes	server-side password validation

Schema map — security policy and updates

Plugin defaults and the policies intentionally left to other layers.

Setting key or owner	Default	Core hook / authority
remove_version	no	wp_head / the_generator
security_headers	yes	wp_headers
frame_options	SAMEORIGIN	wp_headers
disable_ai_connectors	yes	wp_supports_ai / Connectors screen
core_update_policy	minor	automatic core-update filters
Translation files	inherit	WordPress / host / fleet tooling
Plugin and theme code	per-item	WordPress per-item choices
WP_AUTO_UPDATE_CORE	operator	wp-config.php wins

Schema map — content policies

The schema group is authoritative; emoji removal is a Content setting.

Setting key or owner	Default	Core hook / authority
disable_comments	yes	comments, UI, post-type support
disable_pingbacks	yes	default ping options
disable_self_pingbacks	yes	pre_ping
disable_author_archives	yes	template_redirect
redirect_attachment_pages	yes	template_redirect
disable_emojis	yes	init removes emoji assets
limit_unfiltered_html_to_admins	yes	user_has_cap drops the cap for non-admins
disable_post_passwords	no	CSS hides the editor's password option
force_classic_editor	no	four editor gates answered false

Schema map — everyday UX and media

Small workflow defaults stay independent from content policy.

Setting key or owner	Default	Core hook / authority
lowercase_upload_filenames	yes	sanitize_file_name at priority 20
media_sizes_panel	yes	read-only meta box on attachments
title_only_admin_search	no	post_search_columns
frontend_admin_bar_behavior	''	show_admin_bar

Schema map — login, branding, and performance

Schema keys live in one option array; constants remain above plugins.

Setting key or owner	Default	Core hook / authority
disable_remember_me	no	login UI / cookie expiration
session_regular_days	2	auth_cookie_expiration
remember_me_days	14	auth_cookie_expiration
login_logo_behavior	keep_default	login header presentation
mail_deliverability_notice	yes	admin_notices when the From address looks undeliverable
throttle_heartbeat	no	Heartbeat settings / enqueue
wpyeg_better_by_default	array	the only wp_options row
DISALLOW_FILE_EDIT	manual	wp-config.php
revisions / autosave	manual	wp-config.php constants

Somebody will install two of these

Two defaults plugins on one site — or this one plus a security suite doing half the same job.

1. **Nothing errors.** No crash, no warning, no line in a log.

2. **Nothing is clobbered.** Each plugin keeps its own settings safely.

3. **One of them silently stops working.** And its settings screen goes on saying otherwise.

Some settings share. Some fight.

The difference is entirely in what your callback does with the argument it is handed.

1. **Adding to a list.** Everybody's contribution survives. Security headers work this way.

2. **Replacing a value.** Last one wins, the rest are discarded. Session length works this way.

3. **Nothing tells you which you wrote.** Both are add_filter(). Both look identical at the call site.

Check any site in about a minute

WordPress keeps every registered callback in one global. Ask it who is on the hook you care about.

```
// run this inside: wp eval '...'
global $wp_filter; $hook = 'auth_cookie_expiration';
if ( empty( $wp_filter[ $hook ] ) ) { echo "nobody is filtering it\n"; return; }
foreach ( $wp_filter[ $hook ]->callbacks as $prio => $cbs ) {
    foreach ( $cbs as $id => $cb ) { printf( "%-6s %s\n", $prio, $id ); }
}   // each line is one callback; several means shared, not settled
```

Several entries means they share the hook. Whether anyone is losing depends on what the callbacks do — go and read them.

Three attempts. Two were withdrawn.

The useful third design observes real filter runs without pretending it can identify the culprit.

1. **Read the plugin's code for clues.** Named plugins that were doing nothing at all.

2. **Run the setting, see what comes out.** Meant running other people's code to check on them.

3. **Observe real runs at PHP_INT_MAX.** Detects divergence late, without attribution; same-priority callbacks can still run later.

Where they contest a setting, defaults plugins are alternatives.

Two plugins replacing the same value give you one plugin's behaviour and two settings screens — only one of which is telling the truth.

Confirm the overlap first. Then pick the one whose choices you agree with.

Thanks, WPYEG!

Set your defaults wisely.

Files: `sane-defaults.zip` · `wordpress-default-settings.md`

Questions? License GPL-3.0-or-later